

Taller práctico - Bloque 5

Integrantes:

- Brayan Stiver Guerrero Acosta
- Juan David Jurado Paz

Actividad 1:

Elementos	¿Qué relación existe en el problema?	¿Existencia independiente?	Decisión inicial
Estudiante – Préstamo	Un estudiante solicita y tiene préstamos de libros. Se relacionan para colaborar, pero ninguno es parte del otro.	Sí. El estudiante existe sin préstamos. El préstamo es un registro que referencia al estudiante, pero el estudiante no "contiene" al préstamo.	Asociación (Estudiante 1 — 0..* Préstamo)
Libro – Ejemplar	Un libro (título/obra) puede tener varios ejemplares físicos (R10). Cada ejemplar corresponde a un único libro.	Parcial. El ejemplar no tiene sentido sin su libro . El libro como registro de catálogo podría existir sin ejemplares.	Composición (Libro ◆— 1..* Ejemplar), por la regla "cada ejemplar corresponde a un único libro" y porque el ejemplar depende de su libro.
Préstamo – Renovación	El préstamo registra las renovaciones realizadas, conservando fecha anterior, nueva fecha y fecha en que se hizo (R11).	No para la renovación. Una renovación solo existe como parte de un préstamo concreto. El préstamo sí existe sin renovaciones.	Composición (Préstamo ◆— 0..* Renovación)
Usuario – Estudiante	Los estudiantes son un tipo de usuario de la biblioteca (R12). Comparten identificación, nombre y correo.	Un estudiante es un usuario: no son dos objetos que colaboran, sino una especialización.	Herencia (Estudiante —▷ Usuario), con atributos comunes en la superclase.
Usuario – Bibliotecario	Igual que el anterior: el bibliotecario también es un usuario con datos propios adicionales (R12).	Igual: especialización, no objetos separados.	Herencia (Bibliotecario —▷ Usuario)

Preguntas de discusión

1. ¿Que dos clases se relacionen significa necesariamente que una sea parte de la otra?

No. Estar relacionadas solo indica que hay colaboración o dependencia entre ellas. La mayoría de relaciones son **asociaciones simples** (Estudiante–Préstamo)

2. ¿Qué evidencia del requisito utilizaron para decidir?

R10 ("cada ejemplar corresponde a un único libro") → fundamenta la composición Libro–Ejemplar: la dependencia exclusiva de un ejemplar a su libro.

R11 ("un préstamo registra las renovaciones realizadas") → fundamenta la composición Préstamo–Renovación: la renovación se crea, vive y se consulta dentro del préstamo.

R12 ("estudiantes y bibliotecarios comparten identificación, nombre y correo, pero poseen información particular") → fundamenta la herencia.

Actividad 2:

Relación	Tipo seleccionado	Justificación basada en el dominio
Estudiante – Préstamo	Asociación (1 Estudiante — 0..* Préstamo)	Solo necesito expresar que el estudiante <i>realiza</i> préstamos. No hay relación todo–parte: el estudiante existe y tiene sentido sin ningún préstamo, y si un préstamo se elimina, el estudiante sigue en el sistema.
Préstamo – Ejemplar	Asociación (0..* Préstamo — 1 Ejemplar)	El préstamo <i>referencia</i> el ejemplar físico que se entrega. El ejemplar no es parte del préstamo: existe antes y después de él, y a lo largo del tiempo puede aparecer en muchos préstamos. Es colaboración, no pertenencia.
Libro – Ejemplar	Composición (1 Libro ◆— 1..* Ejemplar)	Según R10, cada ejemplar corresponde a un único libro. Hay una relación todo–parte: el ejemplar es una copia física <i>de</i> ese libro. Si el libro se elimina del catálogo, sus ejemplares pierden significado (sin título, ISBN ni autor no se

		pueden identificar). Su ciclo de vida depende del libro.
Préstamo – Renovación	Composición (1 Préstamo ◆— 0..* Renovación)	Por R11, la renovación conserva la fecha anterior de devolución, la nueva fecha y la fecha de realización. Solo tiene sentido dentro del préstamo que renueva. Si el préstamo desaparece, su historial de renovaciones también. Un préstamo puede no tener renovaciones (0..*).

1. **Libro–Ejemplar:** el catálogo no permite ejemplares sin libro asociado ni reasignar un ejemplar a otro libro. Si el equipo asumiera lo contrario (un ejemplar puede sobrevivir al registro del libro), sería **agregación**.
2. **Préstamo–Ejemplar:** asumo que cada préstamo se refiere a **un** ejemplar. Si un mismo préstamo pudiera incluir varios ejemplares, la multiplicidad sería 1..* y algunos equipos lo plantearían como **agregación**. Aun así, yo mantendría asociación, porque el ejemplar no pertenece al préstamo.
3. **Estudiante–Préstamo:** asumo que todo préstamo pertenece a un solo estudiante y que se conserva el historial aunque el estudiante tenga 0 préstamos activos.

Actividad 3:

4. ¿Estudiante es realmente un tipo de Usuario dentro de SmartLibrary?

Sí. El estudiante es una persona que usa la biblioteca: puede identificarse, recibir notificaciones y hacer préstamos. Todo lo que se espera de un Usuario lo cumple un Estudiante

5. ¿Bibliotecario es realmente un tipo de Usuario?

Sí, pero con una salvedad. El bibliotecario también se identifica en el sistema y comparte los datos básicos, por lo que es un Usuario. Sin embargo, su rol es distinto: gestiona el catálogo y los préstamos en lugar de solicitarlos. La herencia es válida porque ambos son usuarios del sistema, pero esa diferencia de comportamiento debe modelarse en sus propias operaciones

6. ¿La existencia de identificación, nombre y correo repetidos sería suficiente para crear la superclase Usuario? ¿Por qué?

No. Repetir atributos es solo una señal, no una justificación. La superclase debe representar un concepto real del dominio con sentido propio (R12 lo establece: ambos son usuarios de biblioteca) y un comportamiento común, por ejemplo, recibir notificaciones.

7. Situación en la que una superclase solo para evitar atributos repetidos sería un diseño débil

Libro y Usuario podrían tener id y nombre/título, pero crear una superclase Entidad con id y nombre solo para ahorrar código no tiene sentido: un libro no es un usuario ni comparten comportamiento. Otro ejemplo: Préstamo y Reserva compartiendo fecha y estado

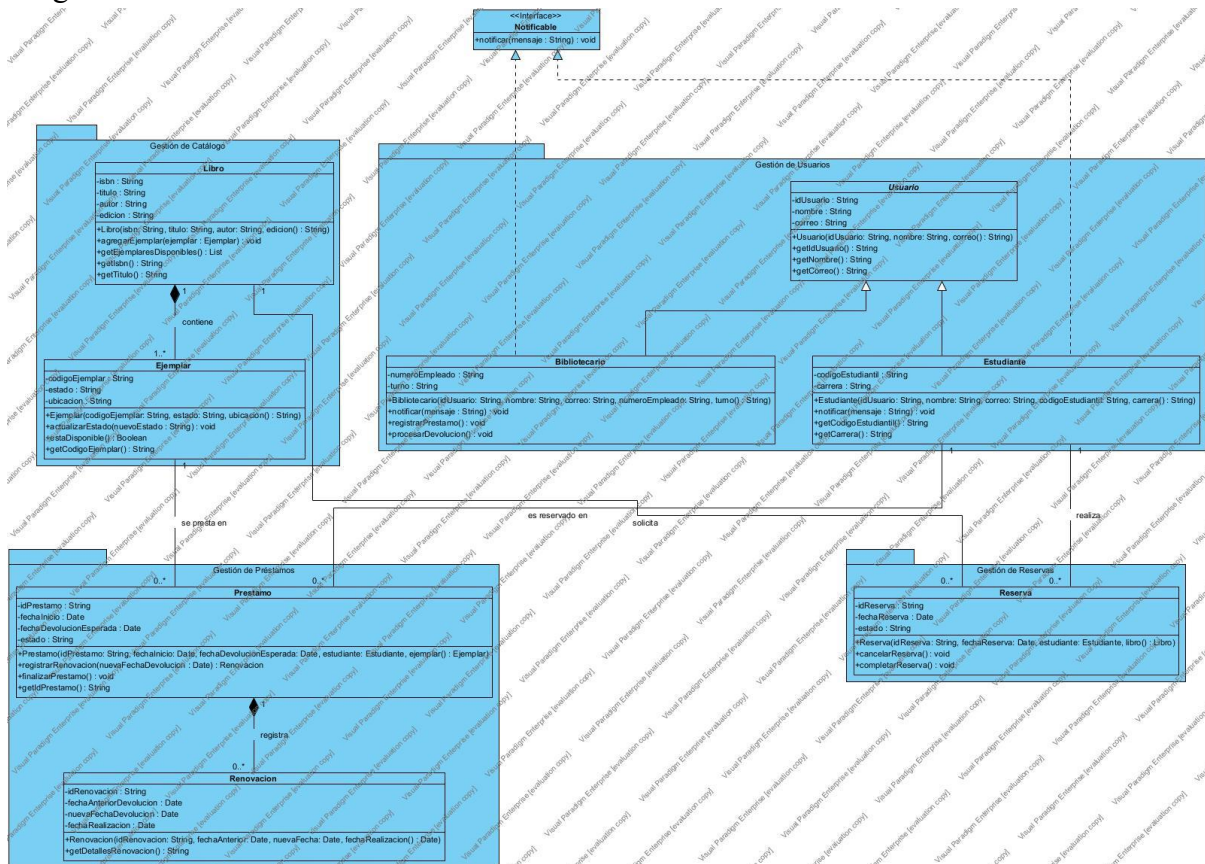
Actividad 4:

1. **Implementan Notificable: Estudiante y Bibliotecario** (las dos subclases de Usuario que reciben notificaciones).
 - **R13** dice que *algunos* usuarios pueden recibir notificaciones, no todos. Por eso Notificable **no** debe implementarla la clase Usuario, porque obligaría a todos sus subtipos a cumplir el contrato.
 - Ambos actores tienen sentido como destinatarios: el estudiante recibe avisos de vencimiento, renovación o reserva disponible, y el bibliotecario avisos operativos (por ejemplo, devoluciones atrasadas).
 - **Supuesto documentado:** si el equipo decide que solo los estudiantes reciben notificaciones, basta con que Estudiante la implemente. Lo importante es justificar la elección con el enunciado y dejarla explícita.
 - Las clases que **no** deben implementarla: Libro, Ejemplar, Prestamo y Renovacion, porque son objetos del dominio que no reciben mensajes, sino que los generan o los provocan.
2. Representación en UML
3. **Garantiza:** que cualquier objeto que la implemente ofrece la operación `notificar(String mensaje)` y puede ser invocado con un mensaje de texto, sin importar qué clase concreta sea.
No especifica:
 - el **medio** de envío (correo, SMS, push, pantalla);
 - el **contenido o formato** del mensaje más allá de ser un String;
 - **cuándo** se notifica ni quién dispara la notificación;
 - **qué pasa si falla** el envío (excepciones, reintentos);
 - atributos o estado interno de la clase.
4. ¿Por qué una interfaz no equivale a una superclase?
 - La **superclase** expresa una relación «**es-un**» y puede aportar **atributos y comportamiento reutilizable** (por ejemplo, Usuario aporta id, nombre, correo).
 - La **interfaz** expresa una **capacidad o contrato** («puede hacer»): declara qué debe ofrecer, no cómo ni con qué datos. No lleva estado.

- Una clase solo puede heredar de **una** superclase en Java, pero puede **implementar varias** interfaces.
- Clases sin relación jerárquica pueden cumplir el mismo contrato: Estudiante y Bibliotecario comparten Usuario, pero mañana cualquier otra clase (por ejemplo, un Administrador o un servicio externo) podría ser Notificable sin ser un Usuario.

Pregunta	Respuesta del equipo
¿Qué promete una clase que implementa Notificable?	Promete ofrecer la operación notificar(String mensaje), es decir, que puede recibir un mensaje de notificación. Quien la use sabe que puede invocarla sin conocer la clase concreta.
¿La interfaz define cómo se envía el mensaje?	No. Solo define <i>qué</i> se puede hacer (firma de la operación), no <i>cómo</i> . Cada clase implementadora decide el mecanismo (correo, SMS, consola, etc.).
¿Puede una clase heredar de Usuario e implementar Notificable al mismo tiempo? Explique.	Sí. Son mecanismos distintos y compatibles: extends aporta la estructura y el comportamiento heredado, e implements añade el contrato. En Java: public class Estudiante extends Usuario implements Notificable. Se permite porque se puede heredar de una sola clase, pero implementar varias interfaces.

Diagrama de clases



Actividad 5: Código completo en repositorio: <https://github.com/bravanckr/Taller-5-patrones-software>

Actividad 6:

Componente	Clases relacionadas	¿De qué otro componente necesita información? ¿Para qué?
Gestión de Usuarios	Usuario, Estudiante, Bibliotecario, Notificable	No depende de otros componentes en esta vista: es proveedor de información. Los demás le piden datos del usuario (identificación, nombre, correo) y le envían las notificaciones.
Gestión de Catálogo	Libro, Ejemplar	Préstamos: para saber si un ejemplar está prestado o disponible antes de modificarlo o retirarlo del catálogo.
Gestión de Préstamos	Prestamo, Renovacion	Usuarios: para identificar al estudiante que solicita el préstamo y notificarle vencimientos y renovaciones. Catálogo: para verificar que el ejemplar existe y registrar cuál se presta. Reservas: para consultar si hay reservas pendientes antes de aprobar una renovación.
Gestión de Reservas	Reserva	Usuarios: para saber quién reserva y a quién avisar cuando el libro esté disponible. Catálogo: para identificar el libro o ejemplar reservado. Préstamos: para saber cuándo se devuelve un ejemplar y poder atender la reserva.

Diagrama de componentes:

